Project 2: CMOS Combinational Logic Gate Design

By: Vy Tang

ID: 77659006

2/14/26

Introduction

The objective of this project is to design and implement a CMOS logic gate in Cadence Virtuoso using both schematic capture and layout, using a given a Boolean equation: $G = \overline{(A + B + C) \cdot D \cdot (E + F)}$. To achieve this, we implemented a static complementary static CMOS design consisting of a Pull-Up Network (PUN) and Pull-Down Network (PDN). After creating PUN and PDN, we would verify our logic design by creating a testbench and run a transient simulation to 800ns at different input pulse widths. Finally, we would analyze the parasitic effects of the logic gates on the output and observe the changes.

Theory

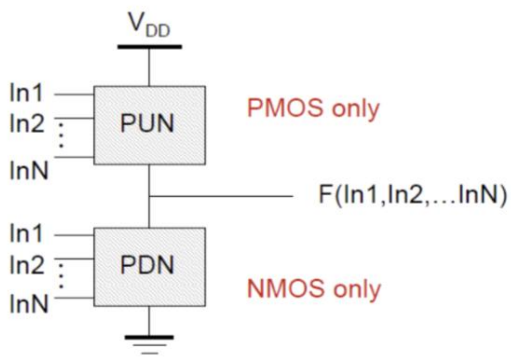

Figure 1: Static CMOS logic topology.

In order to design a static CMOS logic gate, the system needs a PUN and a PDN. A pull-up network consists of only PMOS while a pull-down network consists of only NMOS transistors as illustrated in figure 1. For the PUN, the PMOS transistors source pin will be tied to V_{DD} while for the PDN, the source of the NMOS is tied to ground. These networks work in tandem to form a CMOS logic gate with any given behavior. To design this network, the rule of thumb is always start designing the PDN first.

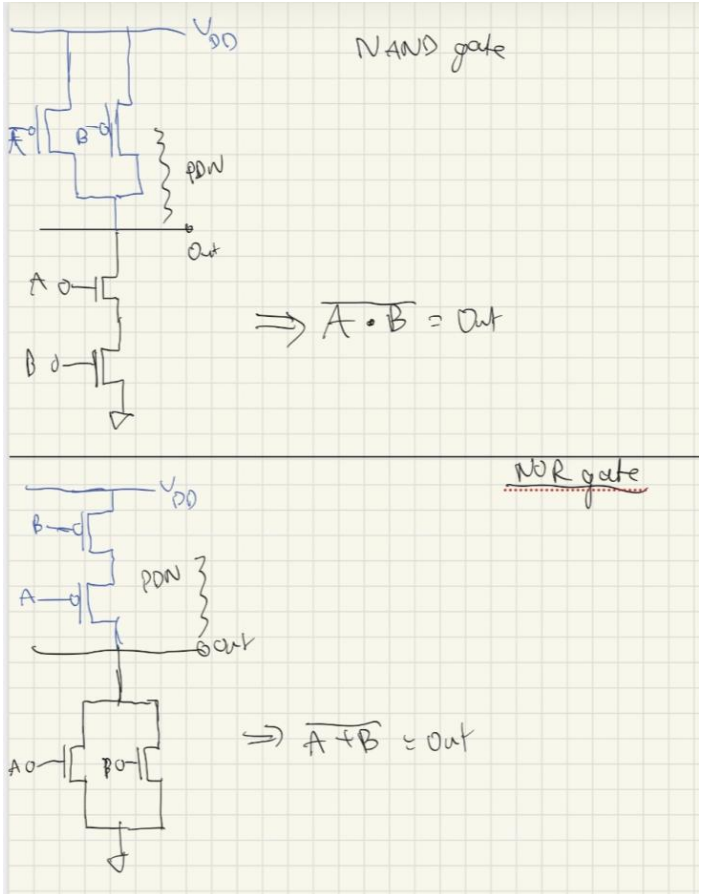

Figure 2. NAND and NOR gate PDN

For the PDN, we first observe the Boolean equation and assess whether to do a NOR or a NAND depending on the equation. If you have an equation with multiplication, then, a NAND should be implemented where the NMOS is placed in series. On the other hand, if there is an addition operation required in the Boolean equation, we would implement a NOR gate, putting two NMOS transistors in parallel to each other. After building the PDN, to construct the PUN, simply do the opposite operation of the PDN. For example, if the PUN consists of NMOS in series, the PUN will have PMOS in parallel. As shown in figure 2 for the NAND and NOR gates, if the PUN has two NMOS in series, then the PDN would have two PMOS in parallel and also vice-versa as shown in the NAND and NOR gate configuration.

Design and Questions

1. Circuit Drawing

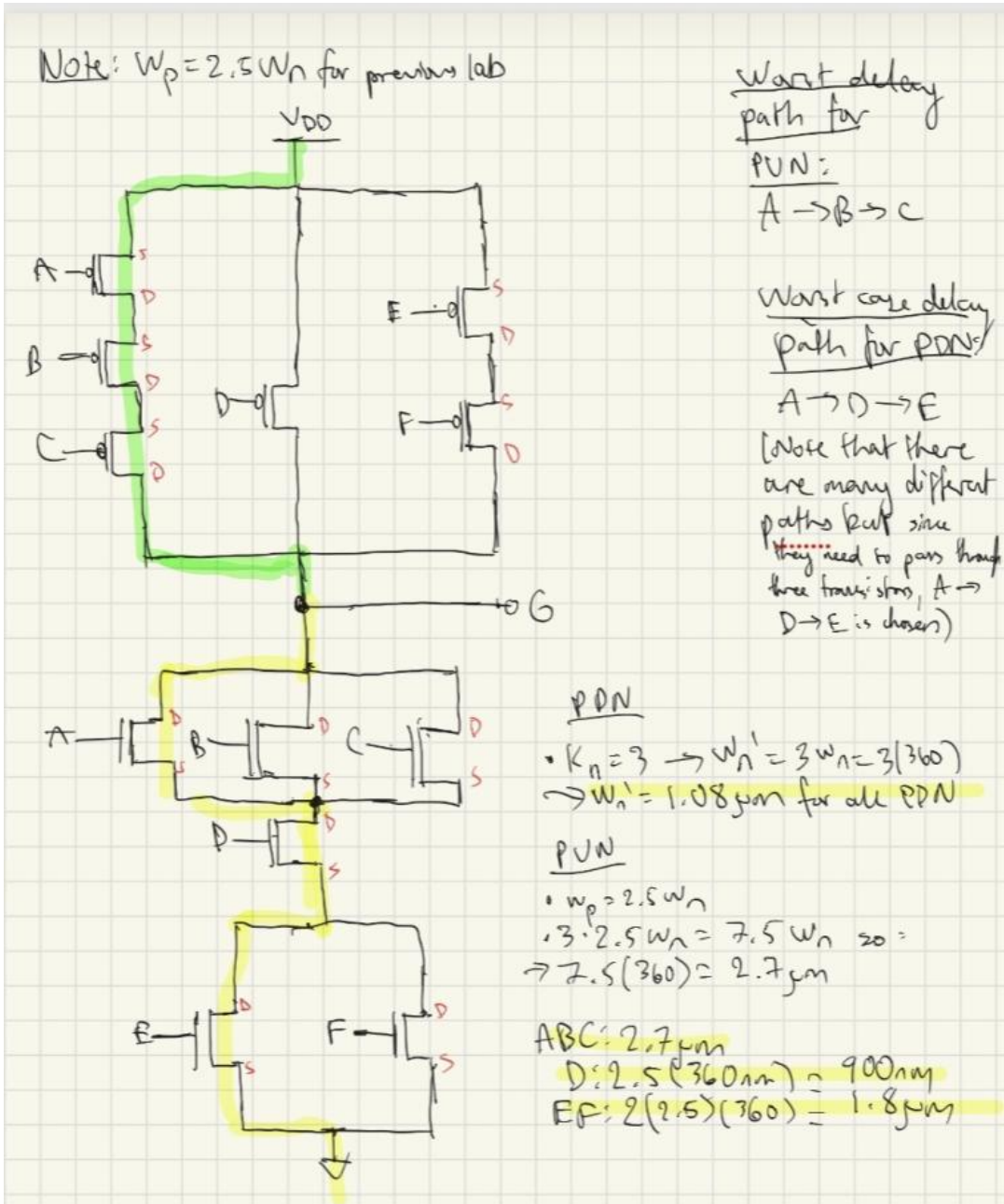

Figure 3. Circuit Drawing of the CMOS Logic circuit

To build the PUN, I identified that immediately, (A+B+C) and (E+F) is a OR operation and hence I put three NMOS in parallel with inputs A,B,C and two in parallel for E and F.

Since they are all multiplied by D, I put an NMOS in series with the two parallel networks to signify that the input D is multiplying (A+B+C) and (E+F). Since A,B,C,D,E, and F were in parallel, for the PDN, they are to be in series and in parallel with input D since input D was in series with both A,B,C and E,F transistors.

In order to find the optimal width for the gate delay, I first have to determine the worst-case scenario. To find the worst-case scenario, I have to identify the longest series conduction path in both the PDN and PUN. According to figure 3, the worst-case scenario for the PUN is through three PMOS transistors in series A → B → C while for the PUN, it also has to pass through 3 NMOS transistors in series albeit it could be going through multiple paths. For this case, I determined that the worst-case scenario for the PUN is the path from A → D → F. Now, we can determine the width of the PDN. For an NMOS with width W_n and resistance R_n, the resistance is inversely proportional to the width and so if there are three NMOS in series, the resistance would be $3R_n$, making the NMOS 3 times slower than the unit inverter. To match this resistance, the width must now be $3W_n$ where each resistance of the NMOS in the PDN is $3 * (R_n/3) = R_n$. So now, the width of the PDN would be 3* 360 nm = 1.08um. For the PUN, since the unit inverter of the $W_p = 2.5W_n$, used from last lab, we would apply the same principles. For path A→B→C, since there are three PMOS in series, the width of the PMOS would be $3 * 2.5W_n$, D path would yield in $2.5W_n$, and E→F path would have $2*2.5W_n$ which results in 2.7um for the width of the A,B,C PMOS transistors, 900nm for the D PMOS transistor, and 1.8um for E,F PMOS transistors respectively.

2. Schematic Design

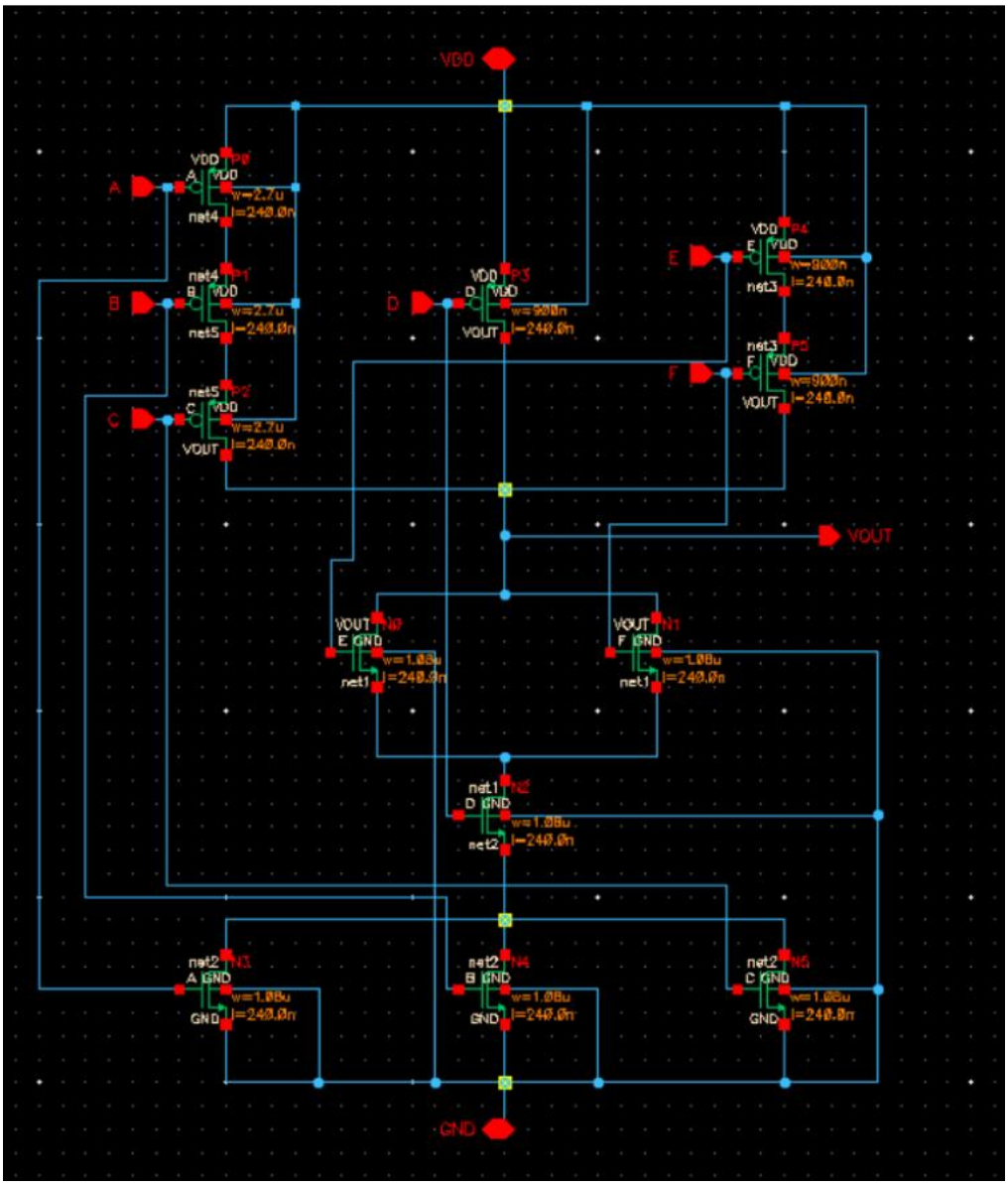

Figure 4. Schematic of CMOS logic

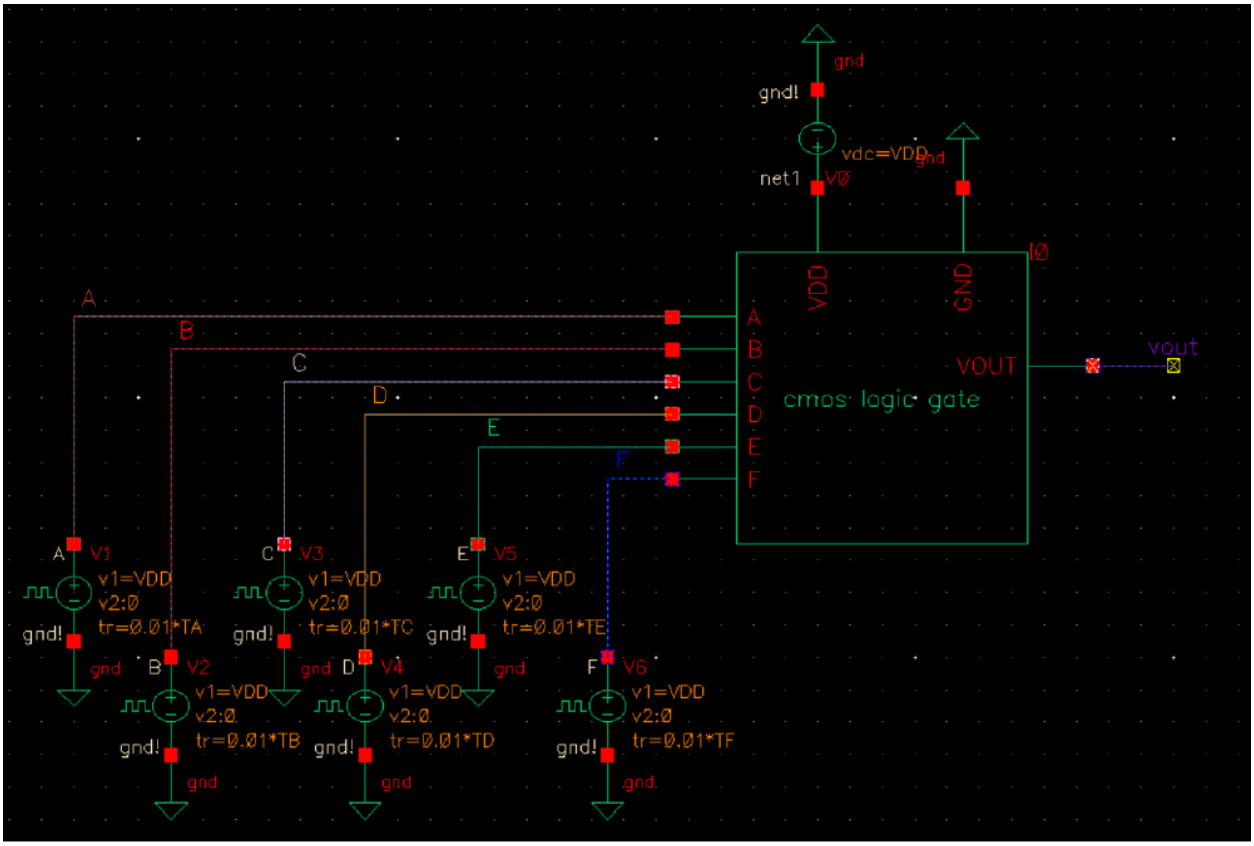

Figure 5. CMOS logic testbench

3. Logic Verification

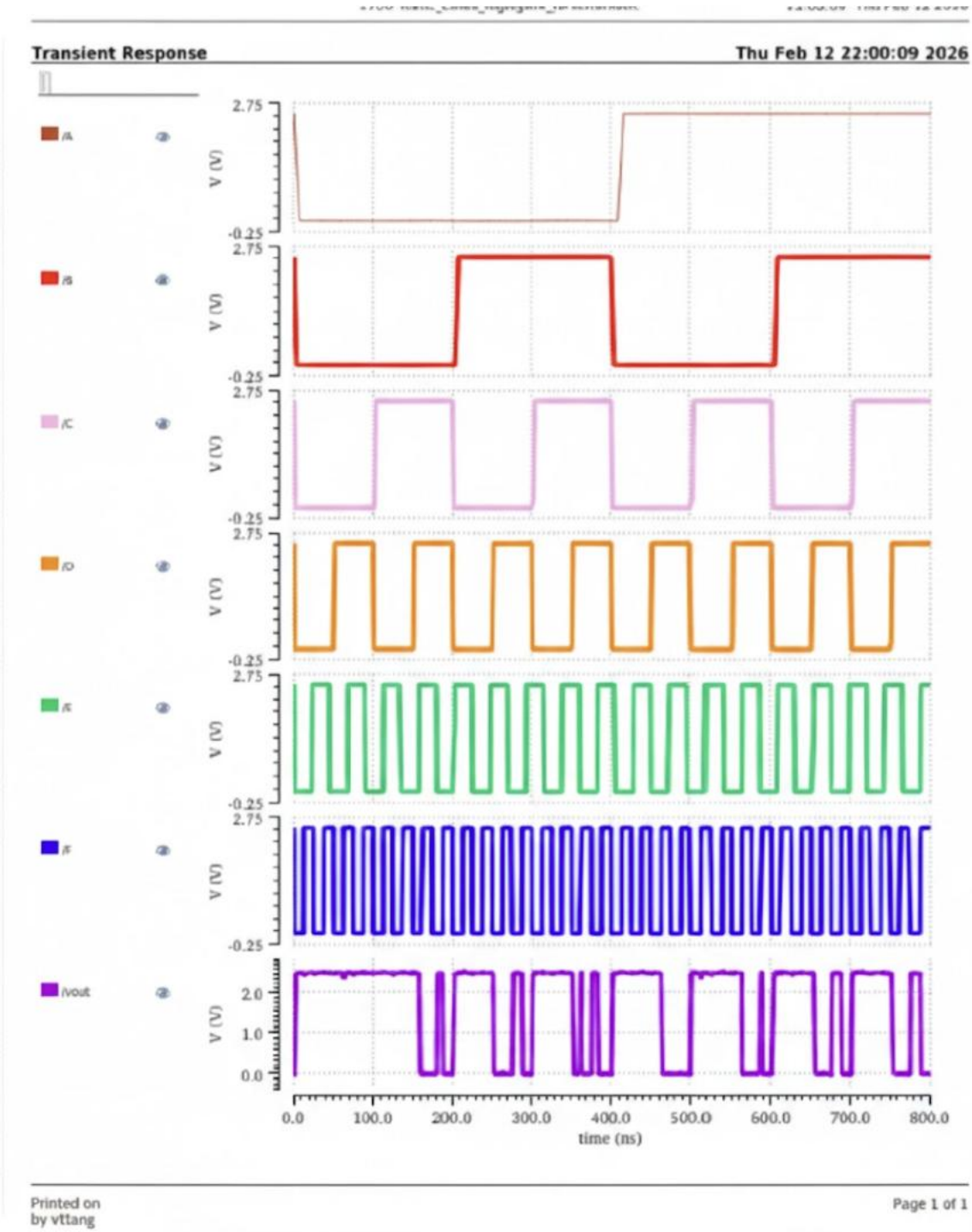

Figure 6. Transient response of CMOS logic with different pulse widths

4. Layout Design

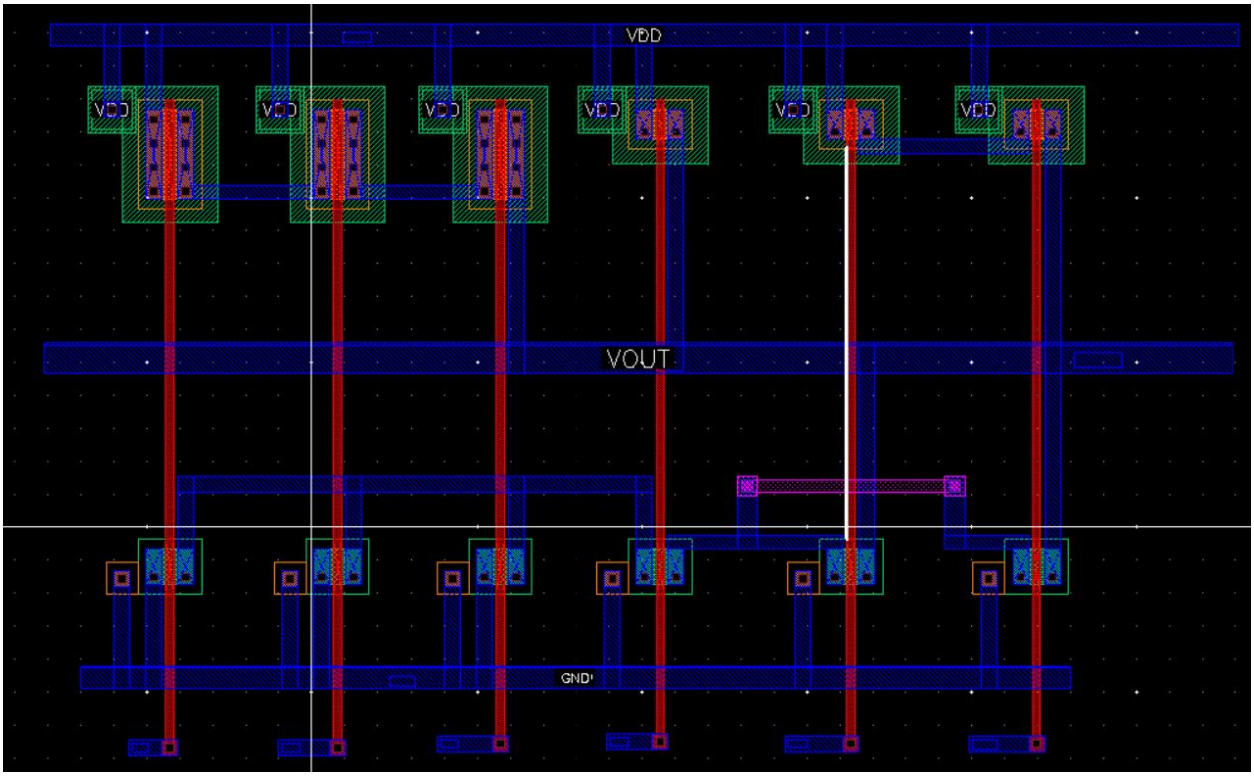

Figure 7. CMOS logic layout

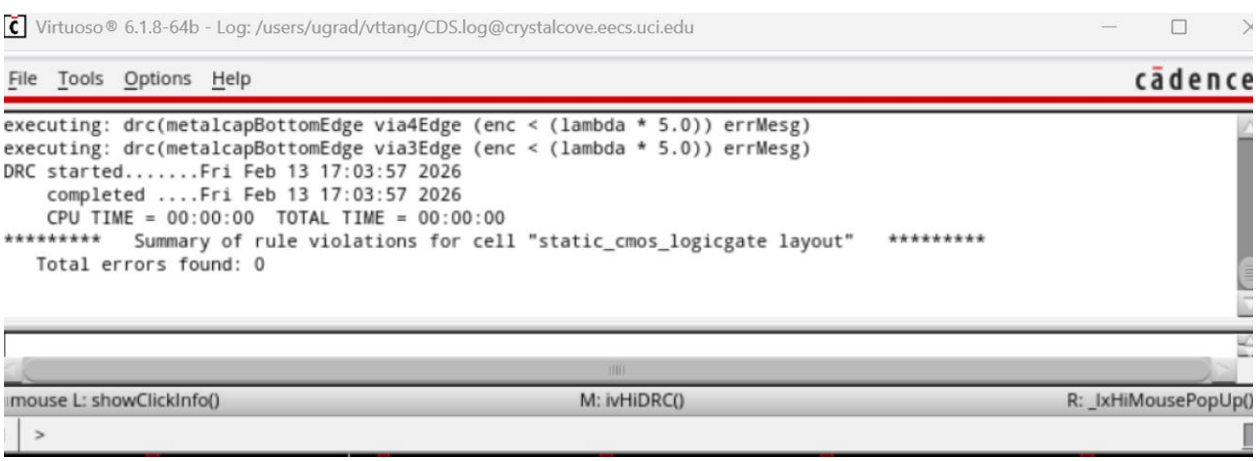

Figure 8. DRC check

```
Command line: /ecelib/eceware/cadence/ic618/tools.lnx86/dfII/bin/64bit/LVS -dir /users/ugrad/vttang/EE
Like matching is enabled.
Net swapping is enabled.
Using terminal names as correspondence points.
Compiling Diva LVS rules...

    Net-list summary for /users/ugrad/vttang/EECS170D/Cadence6/LVS/layout/netlist
       count
        14              nets
        9               terminals
        6               pmos
        6               nmos

    Net-list summary for /users/ugrad/vttang/EECS170D/Cadence6/LVS/layout/netlist
       count
        14              nets
        9               terminals
        6               pmos
        6               nmos


    Terminal correspondence points
    N12       N0        A
    N11       N1        B
    N10       N2        C
    N9        N3        D
    N8        N4        E
    N7        N5        F
    N5        N7        GND
    N13       N6        VDD
    N6        N8        VOUT

Devices in the rules but not in the netlist:
        cap nfet pfet nmos4 pmos4

The net-lists match.

                              layout  schematic
                                 instances
        un-matched               0       0
        rewired                  0       0
        size errors              0       0
        pruned                   0       0
        active                   12      12
        total                    12      12

                                   nets
        un-matched               0       0
        merged                   0       0
        pruned                   0       0
        active                   14      14
        total                    14      14

                                 terminals
        un-matched               0       0
        matched but
        different type           7       7
        total                    9       9
```

Figure 9. LVS check

5. Post-Layout Extraction & Simulation

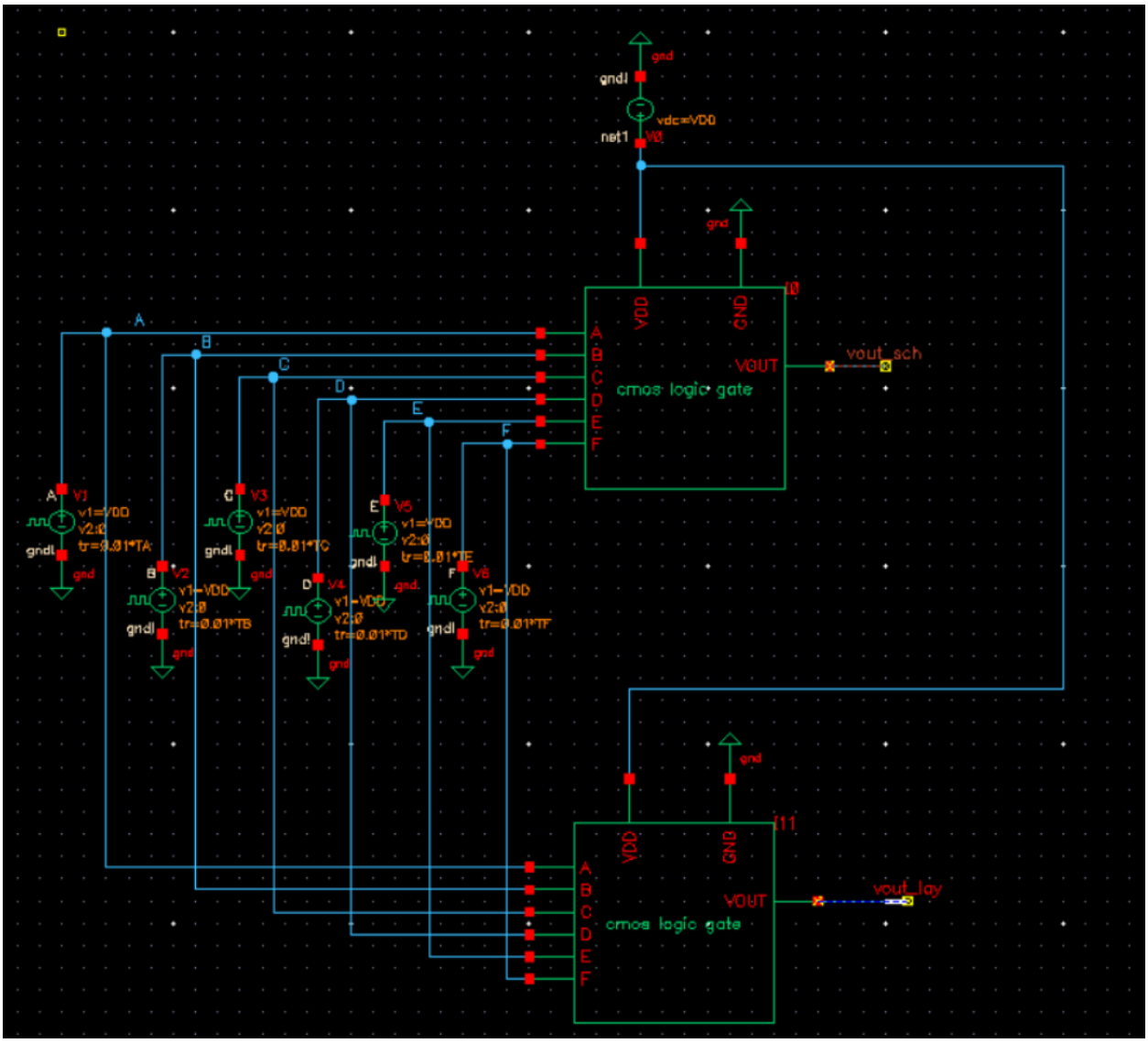

Figure 10. CMOS Logic schematic vs. layout testbench

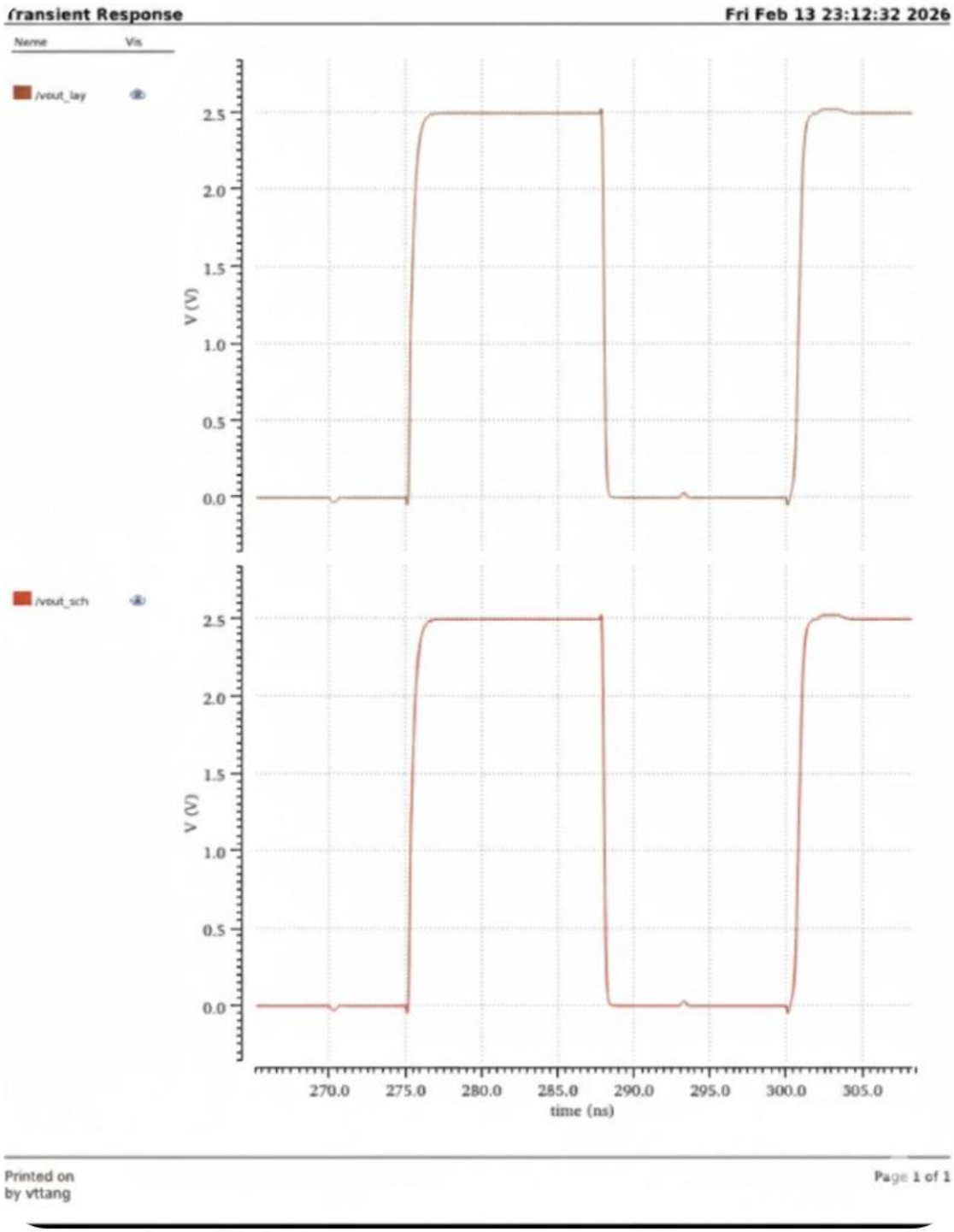

Figure 11. Transient response of Vout of CMOS logic with one being parasitic and one ideal

As seen in figure 11, the layout and the schematic transient responses at the output node are quite similar. However, if zoomed in, we can see that there is a slight delay in the rise and fall time between the ideal CMOS logic gate and the parasitic one. The reason for

this delay is because with parasitic capacitance, it often takes longer to charge and discharge, thus contributing to the delay in the rise and fall time of the output node voltage.

6. Conclusion

In conclusion, in this lab, we were able to successfully implement a static CMOS logic network with the behavior of: $G = \overline{(A + B + C) \cdot D \cdot (E + F)}$. By drawing the pull-up and pull-down network, we were able to find the worst-case scenario delay to find the appropriate width to match the resistance of each transistor so that we can get the optimal gate delay. In addition, we created the testbench schematic to simulate the transient waveforms of each transistor at different pulse-widths and the output matched the expected behavior of the logic function. Finally, we were able to compare and contrast the output waveforms of an ideal and a parasitic CMOS logic function and observed that the one with parasitic capacitance has a slightly lower rising time and falling time compared to the ideal CMOS logic function due to the time it takes to charge and discharge the capacitance.

Bibliography

1) Sedra, A. S. & Smith, K. C, (2014), "*Microelectronic circuits* (7th ed.)", Oxford University Press

2) EECS 170D lecture notes